BLOCKCHAIN-BASED SYSTEMS FOR SECURE OUTSOURCING OF BILINEAR PAIRINGS

Samanvaya Panda
2018121001
B.Tech + M.S(By Research)
Monday 16th November, 2020

***Abstract*—Bilinear pairings are one of the most computational expensive cryptographic primitive. In this project, we propose a blockchain-based secure protocol for outsourcing bilinear pairings computation in the presence of colluding dishonest servers. The advantages of using a blockchain model is that the outsourcer no longer requires to verify the computations performed by the colluding dishonest server. It outsources the verification step as well and reduces its computational burden. We also formally show that it is not efficient to achieve secure outsourcing of bilinear pairings computations without the outsourcer performing exponentiation operation in the presence of colluding dishonest servers. We propose a blockchain-based scheme that can efficiently outsources bilinear pairing computation by reducing exponentiation operation to a few multiplications. Also, our model can easily incorporate any classical scheme which works in the presence of colluding servers. It increases the efficiency of the scheme too by reducing the number of computations the outsourcer is required to perform.**

***Keywords*—Bilinear pairing, Outsourcing computations, Blockchain, Colluding servers, PBFT algorithm**

1. Introduction

1.1. Background

Bilinear pairings are one of the most important cryptographic primitives widely used in signature schemes[1], identity-based encryption schemes [2] etc. Most widely used bilinear pairing constructions are the Weil pairings [3] and Tate pairings [4], [5]. These are based on algebraic curves which makes them computationally very expensive. Hence their usage becomes infeasible for resource-constrained devices like smart cards, sensors. This has prompted researchers to develop novel mechanisms for securely outsourcing computations from a resource-constrained device to a cloud server. These mechanisms facilitates the resource-constrained devices to obtain the actual computations of bilinear pairings from the server without revealing their inputs to it and thus reducing their computational burden.

Hohenberger and Lysyanskaya [6] first proposed a model to securely outsource modular exponentiations in the presence of two non-colluding dishonest servers. Kilian([7], [8]) proposed a protocol for outsourcing computations of any function using interactive proofs so that the outsourcer could verify the computations in polynomial time it receives from the server. Gennaro et al.[9] proposed a non-interactive solution for the same using a fully homomorphic encryption scheme and Yao's Garbled circuit construction where the outsourcer needed to perform some expensive pre-processing.

In this project, we restrict ourselves to outsourcing bilinear pairing computations. Chevallier-Mames et al. proposed the first outsourcing scheme in [10] in one untrusted server model. The main drawback of their scheme was that the outsourcer still needed to perform expensive computations like point multiplication and exponentiation which is not desired. Chen et al. scheme[10] overcame this drawback by introducing a model similar to in [6]. This involved two non-colluding dishonest servers.This scheme reduced the computations performed by the outsourcer significantly as the outsourcer only required to perform multiplications. But the computations could be verified only half of the time in Chen et al's scheme. Other schemes improved upon the verifiablity of computations like the Tian et al. scheme [11] by adding more number queries. The Ren et al. scheme [12] added extra communication rounds to their scheme that provides complete verifiability of computations performed by the server. However, all the schemes required the existence of two non-colluding dishonest server which is not an ideal scenario in real world. The Dong-Ren introduced a new scheme in [13] that works in a single untrusted server model performing exponentiation by small numbers.

1.2. Contribution

In this project, we propose a blockchain-based secure protocol for outsourcing bilinear pairings computation in the presence of colluding dishonest servers. Our proposed scheme uses a modified version of the Dong-Ren scheme. In our scheme, the outsourcer need not verify any computation as verifiability is ensured by blockchain network. This is a major computational advantage over other outsourcing schemes as the outsourcer needed to perform some computation to verify the results obtained from the server. So in our scheme the resource-constrained device outsources both computations as well as the verification phase. We also show that in presence of colluding dishonest servers, it is not efficient for any resource-constrained device to obtain the actual result from the results derived from servers without performing exponentiation operation. This provides a valid intuition for using blockchain-based schemes which reduce the computational burden of the outsourcer by eliminating the verification step. Our blockchain model is quite flexible as well which could

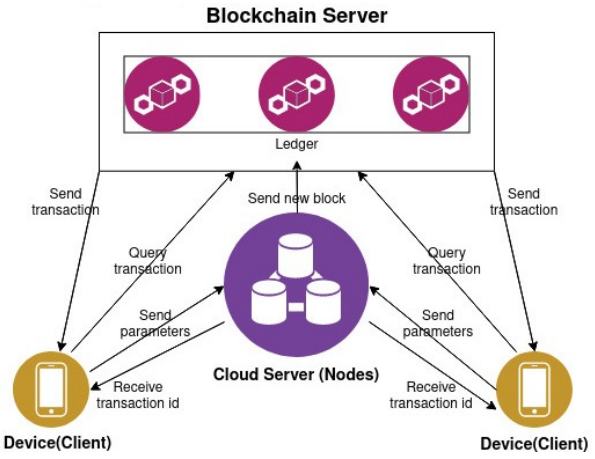


Figure 1. Network model

integrate other schemes provided that they work in the single dishonest server model.

1.3. Network Model

Our proposed model uses a permissioned blockchain. Clients could be thought of as resource-constrained outsourcer T and nodes in the blockchain network could be cloud servers[1]. The outsourcer T would send all the queries to any cloud server U(preferably closet) connected to the network. The cloud server U performs the bilinear pairing computations on the received queries and creates a transaction for the same. Then it broadcasts the transactions to its peer nodes to validate the transaction and also sends the transaction id back to the outsourcer T for its future reference.

Other nodes in the network validate the transaction by verifying the computations made by the cloud server U. Once the transaction is validated and added to the public ledger, the outsourcer T can obtain the results from the ledger using the transaction id provided by U. Finally, the outsourcer T performs some multiplications to obtain desired bilinear pairing computation $e(A, B)$. An overview of our network model could be found in fig. 1.

2. Literature Review

2.1. Preliminaries

Let G_1, G_2 be two additive group and G_T be a multiplicative group both of order p where p is prime number.

Definition 1. *A bilinear pairing on G_1, G_2 and G_T is defined as:-*

$$e : G_1 \times G_2 \rightarrow G_T$$

where they satisfy the following conditions:-

1) *Bilinearity: $\forall A, B \in G_1$ and $\forall C \in G_2$, $e(A + B, C) = e(A, C)e(B, C)$. This implies that, $e(A, kB) = e(kA, B) = e(A, B)^k$.*
2) *Non-degeneracy: There exists $P \in G_1$ and $Q \in G_2$ such that $e(P, Q) \neq 1$*
3) *Computablity: e is efficiently computable.*

Definition 2 (Computational expensiveness of an operation)**.** *Let A_1, A_2 be two operations and $\mathbf{Alg}_1, \mathbf{Alg}_2$ be two worst-case polynomial time algorithms that are the fastest known algorithms to perform those operations respectively. Then, the operation A_1 is said to be computational expensive than A_2 i.e $A_1 > A_2$ iff $\mathbf{Alg}_1 > \mathbf{Alg}_2$.*

2.2. Chevallier-Mames et al. scheme

In this scheme, the groups G_1, G_2 and G_T of order p are made public. Let $A \in G_1$ and $B \in G_2$ be the input points for which $e(A, B)$ must be computed. Let $P \in G_1, Q \in G_2$ be two points for which the outsourcer T has pre-computed the value $e(P, Q)$. The algorithm is as follows:-

1) The outsourcer T picks $a, b \in \mathbb{Z}_p$ and makes three queries to the server:-
$$\alpha_1 = e(A + aP, B), \quad \alpha_2 = e(A, B + bQ),$$
$$\alpha_3 = e(A + aP, B + bQ)$$
2) The outsourcer T checks if $\alpha_i \in G_T$ and computes $(\alpha_i)^p \ \forall i = 1, 2, 3$ else it outputs $\perp$ and halts.
3) Then the outsourcer T computes the value of $e(A, B)$ as:-
$$e(A, B) = \alpha_1^{-b} \cdot \alpha_2^{-a} \cdot \alpha_3 \cdot e(P, Q)^{ab}$$
4) The outsourcer T again picks four random values $a_1, a_2, r_1, r_2 \in \mathbb{Z}_p$ and queries:-
$$\alpha_4 = e(a_1 A + r_1 P, a_2 B + r_2 Q)$$
5) Finally the outsourcer T computes:-
$$\alpha_4' = e(A, B)^{a_1 a_2} \cdot \alpha_1^{a_1 r_2} \cdot \alpha_2^{a_2 r_1} \cdot e(P, Q)^{r_1 r_2 - a_1 a r_2 - a_2 b r_1}$$
and compares $\alpha_4' = \alpha_4$. If they are equal, it outputs $e(A, B)$ else it outputs $\perp$.

2.3. Chen et al. scheme

In this scheme, the outsourcer T uses a *Rand* oracle as mentioned in [6]. This generates a six-tuple $(V_1, V_2, v_1V_1, v_2V_2, v_2V_1, e(v_1V_1, v_2V_2))$ where $V_1, V_2 \in G_1$ and $v_1, v_2 \in \mathbb{Z}_p$. The outsourcer T queries two non-colluding untrusted servers U_1, U_2. The algorithm is as follows:-

1) The outsourcer T runs the *Rand* oracle to generate a six-tuple (V_1, V_2, v_1V_1, v_2V_2, v_2V_1, $e(v_1V_1, v_2V_2)^{-1}$) and another 2 six-tuples (X_1, X_2, x_1X_1, x_2X_2, x_2X_1, $e(x_1X_1, x_2X_2)$) and (Y_1, Y_2, y_1Y_1, y_2Y_2, y_2Y_1, $e(y_1Y_1, y_2Y_2)$).

1. We don't explicitly mention the cloud servers as "dishonest" because majority of the nodes in a blockchain network are honest and dishonest nodes are automatically abandoned by the network. However, the cloud server is not required to behave honestly with the outsourcer and further analysis is done in section 4.3.

2) The outsourcer T queries U_1 in random order:-

$U_1(A + v_1V_1, B + v_2V_2) \rightarrow e(A + v_1V_1, B + v_2V_2) = \alpha_1$;
$U_1(v_1V_1 + v_2V_1, V_2) \rightarrow e(V_1, V_2)^{v_1+v_2}$;
$U_1(x_1X_1, x_2X_2) \rightarrow e(x_1X_1, x_2X_2)$;
$U_1(y_1Y_1, y_2Y_2) \rightarrow e(y_1Y_1, y_2Y_2)$;

3) Similarly the outsourcer T queries U_2 in random order:-

$U_1(A + V_1, -v_2V_2) \rightarrow e(A + V_1, -v_2V_2) = \alpha_2^{-1}$;
$U_1(-v_1V_1, B + V_2) \rightarrow e(-v_1V_1, B + V_2) = \alpha_3^{-1}$;
$U_1(x_1X_1, x_2X_2) \rightarrow e(x_1X_1, x_2X_2)$;
$U_1(y_1Y_1, y_2Y_2) \rightarrow e(y_1Y_1, y_2Y_2)$;

4) The outsourcer T checks if $e(x_1X_1, x_2X_2)$ and $e(y_1Y_1, y_2Y_2)$ are same. If they are same then T computes:-
$e(A, B) = \alpha_1\alpha_2^{-1}\alpha_3^{-1}e(v_1V_1, v_2V_2)^{-1}e(V_1, V_2)^{v_1+v_2}$.

2.4. Dong-Ren scheme

This scheme introduced two new algorithms for outsourcing bilinear pairing computations - one for a single pairing and the other one for t-simultaneous bilinear pairing computations. Both algorithms work in single untrusted server model. In this project, we will discuss only the single pairing version which is mentioned as *BPS* algorithm in [13].

In BPS algorithm, the outsourcer T receives random points $(a_1P, a_2P, \cdots, a_{i+3}P, b_1Q, b_2Q, \cdots, b_{j+3}Q)$ where $a_nP \in G_1, \forall n \in \{1, 2, \cdots, i+3\}$ and $b_mQ \in G_2, \forall m \in \{1, 2, \cdots, j+3\}$, random values $\rho = (\rho_2, \rho_3, \cdots, \rho_i) \in \{-1, 1\}^{i-1}$, $\sigma = (\sigma_2, \sigma_3, \cdots, \sigma_j) \in \{-1, 1\}^{j-1}$ and $t_u, t'_u \in \{1, 2, \cdots, s\}$ where $i = j$, s is small integer and $u \in \{1, 2, \cdots, 7\}$. The random points and values are chosen in such a way that they satisfy the following relations :-

$$b_{j+1}Q + t_1 \sum_{E_k \in B_1} \sigma_k E_k + t_2 \sum_{E_k \in B_{12}} \sigma_k E_k + t_3 \sum_{E_k \in B_{13}} \sigma_k E_k = -b_1 Q \quad (1),$$

$$b_{j+2}Q + t_4 \sum_{E_k \in B_2} \sigma_k E_k + t_5 \sum_{E_k \in B_{12}} \sigma_k E_k + t_6 \sum_{E_k \in B_{23}} \sigma_k E_k = -b_1 Q \quad (2),$$

$$b_{j+3}Q + t_7 \sum_{E_k \in B_3} \sigma_k E_k + t_3 \sum_{E_k \in B_{13}} \sigma_k E_k + t_6 \sum_{E_k \in B_{23}} \sigma_k E_k = b_1 Q \quad (3),$$

$$a_{j+1}P + t'_1 \sum_{E_l \in A_1} \rho_l E_l + t'_2 \sum_{E_l \in A_{12}} \rho_l E_l + t'_3 \sum_{E_l \in A_{13}} \rho_l E_l = -a_1 P \quad (4),$$

$$a_{j+2}P + t'_4 \sum_{E_l \in A_2} \rho_l E_l + t'_5 \sum_{E_l \in A_{12}} \rho_l E_l + t'_6 \sum_{E_l \in A_{23}} \rho_l E_l = -a_1 P \quad (5),$$

$$a_{j+3}P + t'_7 \sum_{E_l \in A_3} \rho_l E_l + t'_3 \sum_{E_l \in A_{13}} \rho_l E_l + t'_6 \sum_{E_l \in A_{23}} \rho_l E_l = a_1 P \quad (6)$$

where E_k and E_l are points b_kQ and a_lP which are distributed randomly in sets A and B respectively. The algorithm proceeds as follows:-

1) T invokes Rand to get $e(a_1P, b_1Q), t_u, t'_u$, $\rho = (\rho_2, \rho_3, \cdots, \rho_i), a_1P, a_2P, \cdots, a_{i+3}P$, $\sigma = (\sigma_2, \sigma_3, \cdots, \sigma_j), b_1Q, b_2Q, \cdots, b_{j+3}Q$
2) T queries server U in random order:

$$e(A + a_1P, B + b_{j+3}Q) = \theta_1,$$
$$e(A + a_1P, b_mQ) = \alpha_m, \forall m \in \{2, 3, \cdots, j+2\},$$
$$e(A + a_{i+3}P, B + b_1Q) = \theta_2,$$
$$e(a_nP, B + b_1Q) = \beta_n, \forall n \in \{2, 3, \cdots, i+2\},$$

3) Then the outsourcer T verifies the result obtained from U by computing $\{Q_1, Q_2, Q_3\}$ as mentioned in [13].
4) Finally, T computes the desired value $e(A, B) = Q_1Q_2Q_3 \cdot e(a_1P, b_1Q)$ where:

$$Q_1 = e(A + a_1P, -b_1Q),$$
$$Q_2 = e(A + a_1P, B + b_1Q),$$
$$Q_3 = e(-a_1P, B + b_1Q)$$

2.5. Notions of inputs and outputs

Inputs refer to the queries sent by the outsourcer T to server U and outputs are the results generated by U and sent back to T. An input is said to *secret* if both (A, B) are known to T but not to U[2]. Similarly, an input is said to be *partially secret* if one of the points from (A, B) is publicly known and the other is only known to T but not to U. If both the points (A, B) are public then the input is said to be *public*. A similar notion could also be implied for output $e(A, B)$.

3. Proposed Scheme

In this section, we first argue that the outsourcer needs to perform exponentiation in the presence of colluding dishonest servers to perform outsourcing efficiently(both in terms of computations and memory). This then raises the question that what is the minimum no.of exponentiations the outsourcer T needs to perform ? We propose a blockchain-based scheme similar to the Dong-Ren scheme where the outsourcer performs exponentiation by a small numbers which is almost same as performing multiplications. Our scheme provides complete verifiability of the computations performed by the server which is not in the case of Dong-Ren scheme.

2. Not known is in the sense known with negligible probability

3.1. Security Definitions

Before elaborating our scheme, we define the security notions for our proposed scheme. Since we assume presence of colluding dishonest servers, our security settings are quite similar to [10] rather than [6]. In our scheme, the entities participating are :- a resource constrained outsourcer T and dishonest server(or group of colluding dishonest servers) U. We define $\text{View}_U(A, B)$ as server's view of points A and B after receiving queries from T. Thus, we say that a scheme is secure if it satisifies the following conditions:-

- **Completeness**:- After successful execution of the scheme, the outsourcer sholud obtain $e(A, B)$ except with negligible probability.
- **Secrecy**:- The server should not be able to learn anything about the points A and B. In other words, for any simulator $\mathcal{S}$ of the ideal world, the output of $\mathcal{S}$ should be the same as $\text{View}_U(A, B)$ or they may be computationally indistinguishable i.e $\mathcal{S} \overset{c}{\equiv} \text{View}_U(A, B)$.
- **Correctness**:- The outsourcer must be able detect if the server is trying to cheat and output $\perp$ in that case except with negligible probability.

Definition 3 (α-efficient, secure outsourcing)**.** *An algorithm pair (T, U) is said to be an α-efficient secure outsourcing algorithm if $\forall$ input x, T runs in at most α-multiplicative factor of the time taken by outsourcing algorithm Alg.*

3.2. Analysis of classical schemes in single dishonest server model

The main objective of any outsourcing scheme is that outsourcer should perform significantly less number of computations compared to the server. So out of all the computations performed by the outsourcer, their ordering in terms of computational expense can be done as - Point Multiplication $>$ Modular Exponentiation $>$ Multiplicative Inversion $>$ Multiplication $>$ Point addition $>$ Additive Inversion = Addition = Point Inversion. Our primary concern is to determine which computations the outsourcer is required to perform. This leads us to the following lemma:-

Lemma 1. *In any outsourcing scheme, the least expensive operation performed by the outsourcer is multiplication.*

Proof. This could be proved using contradiction. Suppose the outsourcer T doesn't need to perform multiplication or any other expensive operation. Since the output of bilinear pairings $e(A, B) \in G_T$, the result obtained by querying the server should also $\in G_T$. If outsourcer T doesn't perform any operation expensive than multiplication, that implies that T has already obtained the result $e(A, B)$ from the server. This would mean that the server knows (A, B) which is contradicts the secrecy property. Hence, such an outsourcing scheme is not secure. ■

From Lemma 1, it is clear that the outsourcer T is required to perform some number of multiplications to obtain $e(A, B)$. This acts as an lower bound on the type of operation the outsourcer needs to perform in terms of computational expensiveness. Since multiplication is the least expensive operation outsourcer performs, efficiency of our scheme would be measured by the number multiplications outsourcer is required to perform. We intend to tighten this lower bound by the following lemma.

Lemma 2. *If the outsourcer performs only multiplications then the dishonest server U may retrieve original inputs (A, B) with non-negligible probability($> \frac{1}{p^2}$) or $\text{View}_U(A, B) \overset{c}{\not\equiv} \mathcal{S}$ where p is the order of G_1, G_2 and G_T.*

Proof. We observe that the outsourcer T sends at least one query of the form $U(A + \sum_i a_i P, B + \sum_j b_j Q) = \theta$ to the dishonest server U. In rest of the queries, it sends query of the form $U(-a_n P, B + b_j Q) = \alpha_n, \forall n$ where $a_n = \sum_{k \subset [i]} a_k$ and $U(A + a_i P, -b_m Q) = \beta_m, \forall m$ where $b_m = \sum_{k \subset [j]} b_k$. After obtaining the results, T obtains $e(A, B) = \theta \cdot \prod_n \alpha_n \prod_m \beta_m \cdot e(P, Q)^{-l}$ where $l = \sum_{i,j} a_i b_j - \sum_n a_n b_j - \sum_m a_i b_m$.

For the server U to obtain A, it has to evaluate $\sum_i a_i P$. It could do so by using $-a_n P$'s obtained from other queries and could thus compute A. Similarly, it could also compute B by computing $\sum_j b_j Q$ from queries received. In this way, the server U retrieves original inputs (A, B) with non-negligible probability in polynomial time. ■

Informally, we could say that while querying the server, the outsourcer reveals all the independent parameters(directly or indirectly) to the server that are required to compute $e(A, B)$. This information is leveraged by the dishonest server to retrieve original inputs (A, B). If some parameters are not revealed(kept private) then the outsourcer would need to perform exponentiation because of the bilinearity property. It could be also noted that the inversion operation could be easily outsourced as $e(A, B)^{-1} = e(A, -B)$. This leads us to the following theorem:-

Theorem 1. *In the presence of colluding servers it is not efficient to outsource bilinear computations without performing exponentiation.*

Proof. From Lemma 2 we conclude that the dishonest server U could retrieve the points $e(A, B)$ from the queries it receives if the outsourcer T only performs multiplications. Hypothetically, the outsourcer T could send exponentially many queries to U. In that case, U would have to wait exponentially long to receive all the queries from T. It would then require exponential number of computations to be able to retrieve inputs (A, B). However, such a scheme would not be efficient because it would require T to perform exponential number of multiplications as well. Hence, the outsourcer T is required to perform exponentiation. ■

This is not a very encouraging result as exponentiation is quite expensive requiring $O(\log(p))$ multiplications. The proof of theorem 1 clearly points out that there exists trade-off between memory required and computations performed. The Dong-Ren scheme [13] finds the middle ground by introducing a concept of exponentiation by small number. The value of t_u's and t'_u's in their scheme

are deliberately kept small($s = 2$) and are not sent through queries to the dishonest server(kept secret). The advantage of having such small numbers as exponent is that the exponentiation operation reduces to a few multiplications. This significantly reduces the computational load of the outsourcer T compared to any other scheme in which T is required to perform exponentiations [10]. However, some people may be skeptical of this approach as any small exponent could be easily brute forced by the attacker. This scheme takes care of this issue by introducing enough parameters and relations that make it difficult for the attacker to guess points (A, B). The scheme requires slightly more memory compared to other scheme which is acceptable given that it secure and reduces the number of computations significantly. We use a scheme similar to Dong-Ren scheme in our proposed blockchain-based scheme.

3.3. Blockchain network

We propose a permissioned blockchain network which consists of resource constrained devices that act as outsourcers, cloud servers that perform the outsourcing computations and blockchain server that maintains the ledger and registers entities into the network. Cloud servers form nodes in the blockchain network while the outsourcers could be thought of as clients. Each node in the network maintains a transaction queue which consists a list of all valid transactions that are yet to added to the ledger. The nodes perform PBFT(Practical Byzantine Fault Tolerant) algorithm to achieve consensus. Since bilinear pairing computations are associated with digital signatures schemes, encryption schemes etc, we assume that the nodes(cloud servers) and clients(resource-constrained devices) have been already registered with blockchain center before the computations are required. Also, we assume that a secure channel exists between the blockchain center and entities(nodes and clients) and clients and nodes have a shared secret key.

The network flow of the proposed scheme is as follows:-

1) The client sends all the queries along with the $H(PID||k)$ to one of the nodes in the blockchain network where PID is the public id of the client and k is the shared secret key between client and node.
2) The node after receiving queries authenticates the client and creates a transaction for the same. Inputs of the transaction are the queries. It then computes $\theta_1, \alpha'_n s, \beta'_m s, \theta_2$ for the given queries using the relations mentioned in section 3.4 and puts them as output of the transaction.
3) After that the node sends transaction id(tx_id) back to the client and finally broadcasts the transaction to all its peer nodes. All the peer nodes check validity of the transaction by verifying the computations performed and add a valid transaction to their respective transaction queue.
4) The leader node(determined using some leader selection algorithm) creates a new block using the transactions from its transaction pool and broadcasts it to all its peer nodes in the network to perform the PBFT consensus algorithm.
5) Once the block is validated, the leader nodes sends the block to the blockchain server. The blockchain server then adds the block to the ledger.
6) Once the block is added to the ledger, the client queries the blockchain server to get the desired transaction. Then it computes $e(A, B)$ using the outputs of the transaction.

3.4. Blockchain-based scheme

Our proposed scheme integrates a scheme similar to Dong-Ren scheme[13] into the blockchain model. Since the outsourcer no longer requires to verify computations, we get rid of those extra computations and parameters. While this reduces the number of computations needed to perform, the scheme becomes less secure compared to [13]. To counter this, we increase the value of certain parameters to achieve equivalent security as provided by [13]. Later we show that even after increasing values of certain parameters, our scheme still requires less number of computations.

In our scheme, we divide the sets A and B into two subsets A_1, A_2 and B_1, B_2 respectively rather than three as in [13]. This is helps us to reduce the number of parameters t_u's from seven to three. Also, the number of sets reduces from seven to three. The modified relations satisfied by random points and parameters are:-

$$b_{j+1}Q + t_1 \sum_{E_k \in B_1} \sigma_k E_k + t_2 \sum_{E_k \in B_{12}} \sigma_k E_k = -b_1 Q \quad (1)$$

$$b_{j+2}Q + t_3 \sum_{E_k \in B_2} \sigma_k E_k + t_2 \sum_{E_k \in B_{12}} \sigma_k E_k = b_1 Q \quad (2)$$

$$b_{i+1}P + t'_1 \sum_{E_l \in A_1} \rho_l E_l + t'_2 \sum_{E_l \in A_{12}} \rho_l E_l = -a_1 P \quad (3)$$

$$b_{i+2}P + t'_1 \sum_{E_l \in A_1} \rho_l E_l + t'_2 \sum_{E_l \in A_{12}} \rho_l E_l = a_1 P \quad (4)$$

Our modified scheme proceeds as follows:-

1) The outsourcer T invokes Rand to get $e(a_1P, b_1Q), t_u, t'_u$ where $u \in \{1, 2, 3\}$,
$\rho = (\rho_2, \rho_3, \cdots, \rho_i), a_1P, a_2P, \cdots, a_{i+2}P$,
$\sigma = (\sigma_2, \sigma_3, \cdots, \sigma_j), b_1Q, b_2Q, \cdots, b_{j+2}Q$
2) T queries server U in random order:

$$e(A + a_1P, B + b_{j+2}Q) = \theta_1,$$
$$e(A + a_1P, b_mQ) = \alpha_m, \forall m \in \{2, 3, \cdots, j+1\},$$
$$e(A + a_{i+2}P, B + b_1Q) = \theta_2,$$
$$e(a_nP, B + b_1Q) = \beta_n, \forall n \in \{2, 3, \cdots, i+1\}$$

3) Then the outsourcer T computes $\{Q_1, Q_2, Q_3\}$ from the result obtained from U as mentioned in 4.1.
4) Finally, T computes the desired value $e(A, B) = Q_1Q_2Q_3 \cdot e(a_1P, b_1Q)$ where:

$$Q_1 = e(A + a_1P, -b_1Q),$$
$$Q_2 = e(A + a_1P, B + b_1Q),$$
$$Q_3 = e(-a_1P, B + b_1Q)$$

Transaction ID	tx_{id}
Input	V = $((A + a_1P, B + b_{j+2}Q), \cdots , (a_nP, B + b_1Q))$
Output	ans = $(\theta_1, \alpha_2, \alpha_3, \cdots , \alpha_{j+1}, \beta_2, \beta_3, \cdots , \beta_{i+1}, \theta_2)$
Transaction Hash	TH
Signature	$sign_{tx_{id}}$

TABLE 1. STRUCTURE OF TRANSACTION

Previous Block Hash	PBH
Merkle Tree Root	MTR
TimeStamp	TS
Leader Node	$Node_{L_{id}}$
Signature of Leader Node	$sign_{L_{id}}$
List of Transactions	$tx_i, \forall i = \{1, \cdots , n\}$
Current Block Hash	CBH
Signature of Verifiers	$sign_i, \forall i = \{1, \cdots , 2f_s + 1\}$

TABLE 2. STRUCTURE OF BLOCK

We use the above scheme in our blockchain network. The outsourcer T first calls the *Rand* as used in [14], [6], [13] to obtain $e(a_1P, b_1Q), t_u, t'_u$ where $u \in \{1, 2, 3\}, \rho = (\rho_2, \rho_3, \cdots , \rho_i), a_1P, a_2P, \cdots , a_{i+2}P, \sigma = (\sigma_2, \sigma_3, \cdots , \sigma_j), b_1Q, b_2Q, \cdots , b_{j+2}Q$. Then T executes Algorithm 1 to obtain the result $e(A, B)$. Upon receiving any query, the server node runs Algorithm 2 to create and broadcast the transaction. The structure of a transaction could be found in table 1. Peer nodes in the network upon receiving a new transaction check its validity and add a valid transaction to their transaction queue. It is worthy to note that a transaction doesn't include any information about the client. However, this doesn't imply anonymity as an adversary could figure out the client through network analysis by monitoring the addresses to which transaction id is being sent.

A leader selection algorithm is run among all the nodes of the network to select a leader node. The leader node then performs Algorithm 3 to create a new block and sends it to the blockchain server. The structure of a block can be found in table 2. When a peer node receives a new block, it validates the block by checking if all the transactions present inside the block also exists in its transaction queue. If all the transactions are found then the node sends a message **Valid** to the leader node along with its signature. Once a block is verified and a peer node receives a **Commit** message from the leader node, it deletes all the accepted transactions from its transaction queue. The blockchain upon receiving a new block first verifies the signatures attached with block to check whether the block has been endorsed by at least $2f_s + 1$ nodes where f_s denotes the number byzantine faulty nodes. Verification step performed by blockchain server is essential ensure that there are no forks in the ledger. Any malicious intent of the leader node add an invalid block to the ledger may make entire ledger invalid. Once a block is verified, the blockchain server adds that to the ledger.

Algorithm 1: Algorithm for outsourcer T.

Input: R = $e(a_1P, b_1Q)$
V = Random vector consisting of

$$(A + a_1P, B + b_{j+2}Q),$$
$$(A + a_1P, b_mQ), \forall m \in \{2, 3, \cdots , j + 1\},$$
$$(A + a_{i+2}P, B + b_1Q),$$
$$(a_nP, B + b_1Q), \forall n \in \{2, 3, \cdots , i + 1\}$$

Output: $ans = e(A, B)$
function `outsource(`V, R`)`**:**
  $i \leftarrow 0$
  $ans \leftarrow 0$
  $tx_id \leftarrow NULL$
  while *not ans* **do**
    Send $V, H(PID||k_i)$ to U_i.
    $tx_id \leftarrow U_i.receive()$
    if *tx_id* $\neq$ *NULL* **then**
      $tx \leftarrow Ledger.queryOnId(tx_id)$
      if *tx.input* = V **then**
        $Compute\ Q_1, Q_2, Q_3$
        $ans \leftarrow Q_1Q_2Q_3 \cdot R$
      end
      $i \leftarrow i + 1$
    end
  end
  return ans

Algorithm 2: Algorithm for server node U

function `genTransaction(`V, S`)`**:**
  $S' \leftarrow H(PID||k_i)$
  if $S' = S$ **then**
    $tx = Ledger.queryOnInput(V)$
    if *tx* $\neq$ *NULL* **then**
      Send $tx.id$ to outsourcer T
    end
    else
      $found \leftarrow False$
      for *all tx in transaction queue* **do**
        if $tx.input = V$ **then**
          $found \leftarrow True$
          Send $tx.id$ to outsourcer T
        end
      end
      if *not found* **then**
        $tx \leftarrow createTransaction()$
        $tx.input \leftarrow V$
        $tx.output \leftarrow$
        $(\theta_1, \alpha_2, \alpha_3, \cdots , \alpha_{j+1}, \beta_2, \beta_3, \cdots , \beta_{i+1}, \theta_2)$
        Send $tx.id$ to outsourcer T
        Add tx to transaction queue
        Broadcasts tx to all its peer nodes
      end
    end
  end

Algorithm 3: Algorithm for Leader node.

function `genBlock()`**:**
- $blk \leftarrow createBlock()$
- Broadcast blk to all peer nodes
- $count \leftarrow 0$
- $i \leftarrow 0$
- **while** *count* $< 2f_s + 1$ *and* $i < 3f_s + 1$ **do**
  - $(msg_i, sign_i) \leftarrow U_i.receive()$
  - **if** msg_i = *Valid* **then**
    - $count \leftarrow count + 1$
    - Add $sign_i$ to blk
  - **end**
  - $i \leftarrow i + 1$
- **end**
- **if** *count* $\geq 2f_s + 1$ **then**
  - Send **Commit** messages to all its peer nodes.
  - Send blk to blockchain server.
  - Delete all the corresponding transactions in blk from its transaction queue.
- **end**

4. Analysis of Proposed Scheme

4.1. Correctness

We define Q_1, Q_2, Q_3 in the same way as proposed in original scheme [13], T computes:-

$$
\begin{aligned}
Q_1 = &\left(\prod_{\substack{b_mQ\in B_1\\ \sigma_m=1}} \alpha_m \left(\prod_{\substack{b_mQ\in B_1\\ \sigma_m=-1}} \alpha_m\right)^{-1}\right)^{t_1} \cdot \\
&\left(\prod_{\substack{b_mQ\in B_{12}\\ \sigma_m=1}} \alpha_m \left(\prod_{\substack{b_mQ\in B_{12}\\ \sigma_m=-1}} \alpha_m\right)^{-1}\right)^{t_2} \cdot \alpha_{j+1} \\
&= e(A + a_1P, Q)^{b_{j+1}+t_1 \sum\limits_{E_k\in B_1} \alpha_k E_k + t_2 \sum\limits_{E_k\in B_{12}} \alpha_k E_k} \\
&= e(A + a_1P, -b_1Q)
\end{aligned}
$$

$$
\begin{aligned}
Q_2 = &\left(\prod_{\substack{b_mQ\in B_2\\ \sigma_m=1}} \alpha_m \left(\prod_{\substack{b_mQ\in B_2\\ \sigma_m=-1}} \alpha_m\right)^{-1}\right)^{t_3} \cdot \\
&\left(\prod_{\substack{b_mQ\in B_{12}\\ \sigma_m=1}} \alpha_m \left(\prod_{\substack{b_mQ\in B_{12}\\ \sigma_m=-1}} \alpha_m\right)^{-1}\right)^{t_2} \cdot \theta_1 \\
&= e(A + a_1P, Q)^{b_{j+2}+t_3 \sum\limits_{E_k\in B_2} \alpha_k E_k + t_2 \sum\limits_{E_k\in B_{12}} \alpha_k E_k} \\
&\cdot e(A + a_1P, B) \\
&= e(A + a_1P, B + b_1Q)
\end{aligned}
$$

$$
\begin{aligned}
Q_3 = &\left(\prod_{\substack{a_nP\in A_1\\ \rho_n=1}} \beta_n \left(\prod_{\substack{a_nP\in A_1\\ \rho_n=-1}} \beta_n\right)^{-1}\right)^{t'_1} \cdot \\
&\left(\prod_{\substack{a_nP\in A_{12}\\ \rho_n=1}} \beta_n \left(\prod_{\substack{a_nP\in A_{12}\\ \rho_n=-1}} \beta_n\right)^{-1}\right)^{t'_2} \cdot \beta_{i+1} \\
&= e(P, B + b_1Q)^{a_{i+1}+t'_1 \sum\limits_{E_l\in A_1} \rho_l E_l + t'_2 \sum\limits_{E_l\in A_{12}} \rho_l E_l} \\
&= e(-a_1P, B + b_1Q)
\end{aligned}
$$

Finally,

$$
\begin{aligned}
&Q_1Q_2Q_3 \cdot e(a_1P, b_1Q) \\
&= e(A, B)e(A, Q)^{(-b_1+b_1)}e(P, B)^{(a_1-a_1)} \\
&\cdot e(P, Q)^{(a_1b_1-a_1b_1)} \\
&= e(A, B)
\end{aligned}
$$

4.2. Security analysis

Theorem 2. *The proposed scheme is a secure bilinear outsourcing protocol.*

Proof. To prove that our proposed scheme is secure we need to show that our scheme satisfies the completeness, secrecy and correctness property. The completeness property is achieved in two parts. The first part is achieved between the outsourcer T and server U when T receives a transaction id from server U. If T receives a wrong transaction id(id of some other transaction) or an invalid transaction id which doesn't exists or doesn't receive an id at all then it could query a different server. Even after querying all the servers if T is not able to evaluate $e(A, B)$ implies that all the nodes in the network are dishonest or captured by adversary. But the probability of such an event happening is 2^{-n} which is negligible. The second part is achieved between the server U and its peer nodes. When U creates a valid transaction, it eventually gets added to ledger as majority of the nodes in the network are honest.

To prove the secrecy property, an argument similar to in [13] could be used. Instead of three, server now needs to guess two t_u's and the β_i's are now distributed over three sets. So, the server U could guess both points (A, B) with a probability $s^{-2}6^{-(i-1)}6^{-(j-1)}$. For our chosen parameters, this probability is negligible($\approx 10^{-22}$).

The correctness of the scheme is discussed in section 4.1. Moreover if the sever U tries to cheat by providing wrong values as output of the transaction, it would be detected by other nodes and that transaction would never be added to ledger. Also if the server U creates a transaction with wrong inputs then the transaction would be added to the ledger but outsourcer T would be able to detect that. So, overall our scheme is correct. ■

4.3. Resilience against different attacks

Our proposed scheme is resistant against man-in-the-middle attack as any change made by the adversary either in input or transaction id could be easily detected by the outsourcer. Impersonation attack is also not feasible to mount in our scheme as the attacker does not know the

Scheme	Computations	No.of points
Dong-Ren(with verify)	46 Mul + 4 PA	24
Dong-Ren(without verify)	40 Mul + 4 PA	24
Our Scheme	36 Mul + 4 PA	30

TABLE 3. COMPARISON OF DIFFERENT SCHEME

shared secret key between the outsourcer and server. For similar reasons replay attack is not possible as well. Node capture attack is also not possible for our scheme as none of the parameters are stored in the device.

In our scheme there could be a possibility of duplicate transaction attack where a client could behave maliciously by querying the same inputs to the server U again and again. This could result in blocks having duplicate transactions. To counteract this attack, the server checks the ledger and its transaction queue for transactions with the same input before creating a new transaction. If such an transaction is found then the server simply returns transaction id of that transaction and is not required to create a new transaction. Our scheme is resistant to forking as there is only a single miner in our network. Also invalid transactions do not exist in the blocks which is ensured by the blockchain server preventing blockchain injection attacks. Sybil attacks are also not possible as the network is permissioned and creating new peers would require registration and authentication of nodes.

5. Comparative Study

A major advantage of our scheme compared to other schemes is that the outsourcer need not verify the bilinear computations made by the untrusted server. Verification is done by other nodes present in the blockchain. This reduces the computational load of resource-constrained outsourcer significantly. Compared to schemes like [14], [11], [12] which require two non-colluding servers, our scheme works in the presence of one dishonest server. Since our scheme is quite similar to [13], the resource constrained devices are not required to perform any expensive operations like point multiplication and exponentiation contrast to [10]. So, we compare our scheme with Dong-Ren scheme in table 3.

We could have used the scheme as mentioned in [13]. But this would require the outsourcer to perform unnecessary computations for verifying computations. Even if the outsourcer doesn't verify the computations, there won't be a significant reduction in number of computations as the scheme requires a lot of redundant computations. In our proposed scheme, we remove the extra parameters that were required to verify computations earlier. However, this leads to a less secure scheme. To increase the security of our scheme, we increase the value of $i = j = 14$ and the value of $s = 2$ remains the same. With these parameters, we are able to achieve the probability of guessing the inputs almost equal to that of in [13], [12]($\approx 10^{-22}$). However there is slight increase in the number of points and queries required to which is understandable as it is required to increase the security of the scheme. We also perform four point addition as in [13].

6. Conclusion

Our proposed scheme is the first blockchain-based scheme for outsourcing bilinear pairing computations from a resource-constrained device to a cloud server. An intuitive reason to use blockchain-based schemes is that they eliminate the verification phase. By using blockchain, we outsource the verification phase as well. The outsourcer is assured that all the computations obtained from the server are correct. It only needs to use those computations to get the desired output. In fact any scheme that can be used in one untrusted server model could also be integrated into our blockchain model. Any improvements made in classical scheme in the one untrusted server model would also improve our scheme as well. Our model could also be easily adapted to perform multiple bilinear pairing computations. The outsourcer needs to send all the queries at once to server and derive desired results from corresponding transaction.

7. Future Research Directions

Our blockchain model uses the public ledger as some sort of cache for the outsourcer. The outsourcer searches for appropriate transaction to obtain its result. The cloud server also checks the ledger before creating a new transaction to avoid duplicate transactions. However once the size of the ledger increases, it may become very time consuming for nodes to search a transaction and eventually delay the process of creating a new transaction. So, a better model could designed to avoid duplicate transactions. Also, further analysis could be done to determine the optimal size of the network as the size increases, time taken to arrive at consensus also increases which may delay the bilinear pairing computation on the resource constrained device.

References

[1] D. Boneh, B. Lynn, and H. Shacham, "Short signatures from the weil pairing," in *Advances in Cryptology - ASIACRYPT 2001, 7th International Conference on the Theory and Application of Cryptology and Information Security, Gold Coast, Australia, December 9-13, 2001, Proceedings* (C. Boyd, ed.), vol. 2248 of *Lecture Notes in Computer Science*, pp. 514–532, Springer, 2001.

[2] D. Boneh and M. K. Franklin, "Identity-based encryption from the weil pairing," in *Advances in Cryptology - CRYPTO 2001, 21st Annual International Cryptology Conference, Santa Barbara, California, USA, August 19-23, 2001, Proceedings* (J. Kilian, ed.), vol. 2139 of *Lecture Notes in Computer Science*, pp. 213–229, Springer, 2001.

[3] A. Weil, "Sur les fonctions algébriques à corps de constantes fini," in *Les Comptes rendus de l'Académie des sciences, 210: 592–594*, 1979.

[4] J. Tate, "wc-groups over p-adic fields," in *Séminaire Bourbaki : années 1956/57 - 1957/58, exposés 137-168*, no. 4 in Séminaire Bourbaki, pp. 265–277, Société mathématique de France, 1958. talk:156.

[5] J. Tate, "Duality theorems in galois cohomology over number fields," in *Proceedings of the International Congress of Mathematicians (Stockholm, 1962), Djursholm: Inst. Mittag-Leffler, pp. 288–295, MR 0175892*, 2010.

[6] S. Hohenberger and A. Lysyanskaya, "How to securely outsource cryptographic computations," in *Theory of Cryptography, Second Theory of Cryptography Conference, TCC 2005, Cambridge, MA, USA, February 10-12, 2005, Proceedings* (J. Kilian, ed.), vol. 3378 of *Lecture Notes in Computer Science*, pp. 264–282, Springer, 2005.

[7] J. Kilian, "Improved efficient arguments (preliminary version)," in *Advances in Cryptology - CRYPTO '95, 15th Annual International Cryptology Conference, Santa Barbara, California, USA, August 27-31, 1995, Proceedings* (D. Coppersmith, ed.), vol. 963 of *Lecture Notes in Computer Science*, pp. 311–324, Springer, 1995.

[8] J. Kilian, "A note on efficient zero-knowledge proofs and arguments (extended abstract)," in *Proceedings of the 24th Annual ACM Symposium on Theory of Computing, May 4-6, 1992, Victoria, British Columbia, Canada* (S. R. Kosaraju, M. Fellows, A. Wigderson, and J. A. Ellis, eds.), pp. 723–732, ACM, 1992.

[9] R. Gennaro, C. Gentry, and B. Parno, "Non-interactive verifiable computing: Outsourcing computation to untrusted workers," in *Advances in Cryptology - CRYPTO 2010, 30th Annual Cryptology Conference, Santa Barbara, CA, USA, August 15-19, 2010. Proceedings* (T. Rabin, ed.), vol. 6223 of *Lecture Notes in Computer Science*, pp. 465–482, Springer, 2010.

[10] B. Chevallier-Mames, J. Coron, N. McCullagh, D. Naccache, and M. Scott, "Secure delegation of elliptic-curve pairing," in *Smart Card Research and Advanced Application, 9th IFIP WG 8.8/11.2 International Conference, CARDIS 2010, Passau, Germany, April 14-16, 2010. Proceedings* (D. Gollmann, J. Lanet, and J. Iguchi-Cartigny, eds.), vol. 6035 of *Lecture Notes in Computer Science*, pp. 24–35, Springer, 2010.

[11] H. Tian, F. Zhang, and K. Ren, "Secure bilinear pairing outsourcing made more efficient and flexible," in *Proceedings of the 10th ACM Symposium on Information, Computer and Communications Security, ASIA CCS '15, Singapore, April 14-17, 2015* (F. Bao, S. Miller, J. Zhou, and G. Ahn, eds.), pp. 417–426, ACM, 2015.

[12] Y. Ren, N. Ding, T. Wang, H. Lu, and D. Gu, "New algorithms for verifiable outsourcing of bilinear pairings," *Sci. China Inf. Sci.*, vol. 59, no. 9, p. 99103, 2016.

[13] M. Dong and Y. Ren, "Efficient and secure outsourcing of bilinear pairings with single server," *Sci. China Inf. Sci.*, vol. 61, no. 3, pp. 039104:1–039104:3, 2018.

[14] X. Chen, W. Susilo, J. Li, D. S. Wong, J. Ma, S. Tang, and Q. Tang, "Efficient algorithms for secure outsourcing of bilinear pairings," *Theor. Comput. Sci.*, vol. 562, pp. 112–121, 2015.